Preliminary Round Submission

Claim IQ

Decision Intelligence Platform for Malaysian TPA Claims Adjudication

– SYSTEM ANALYSIS DOCUMENTATION (SAD) –

GROUP: AmaChuuErs

Domain: Domain 2 - AI for Economic Empowerment & Decision Intelligence

Submitted: 26th April 2026

1. INTRODUCTION

1.1 Purpose

This System Analysis Document (SAD) describes the technical architecture, design decisions, and engineering strategy behind ClaimIQ, an AI-powered claims-intelligence platform for Malaysian Third-Party Administrator (TPA) operations and SME panel clinics.

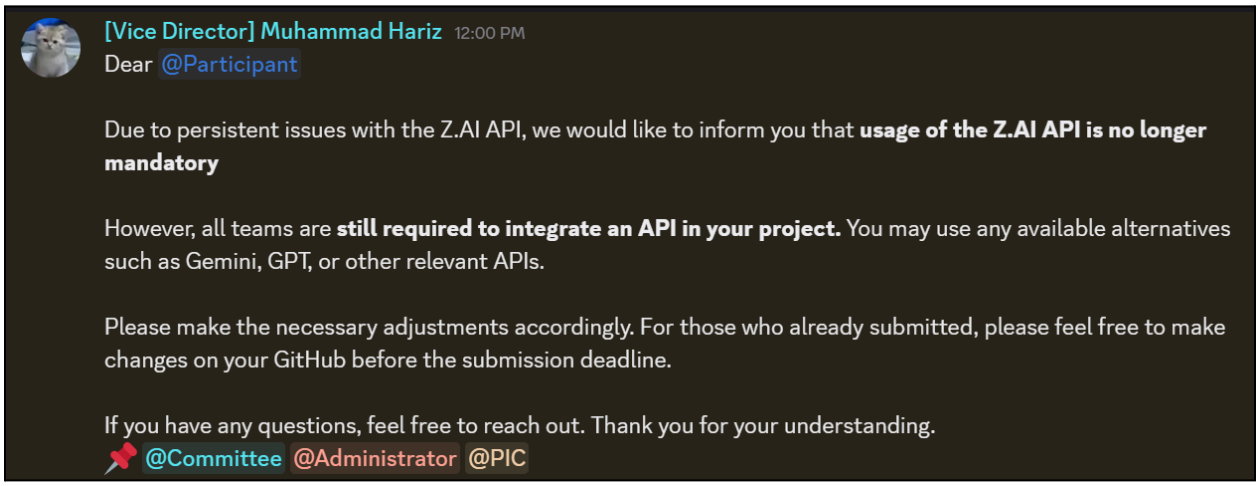
The key elements covered in this document are:

- **Architecture:** A FastAPI-served web application backed by SQLite, with a Large Language Model (LLM) as the central reasoning component across an 8-step claims processing pipeline. Decision-making is gated by deterministic safety rules so the LLM informs but does not autonomously approve or deny claims.
- **Data Flows:** How a claim moves from intake (manual entry, seeded demo, or document upload), through a multi-stage pipeline that combines deterministic validation (Scrubber, Eligibility Engine, Cross-Reference Engine) with LLM-driven reasoning (Extraction, Coding, Adjudication, Fraud Detection, GP Advisory), and back to operators with explainable, audit-logged outputs.
- **Model Process:** The end-to-end workflow for the platform's core decision-intelligence features — adjudication recommendation, fraud anomaly detection, conversational claim Q&A ("Ask GLM about this claim"), bilingual EN/BM advisory generation, and the AI Weekly Intelligence Report.
- **Role of Reference:** This document is the canonical technical reference for development, QA, and the judging panel to understand how ClaimIQ is engineered for explainability, safety, cost-efficiency, and graceful degradation.

Note on the LLM Provider

The hackathon brief originally specified Z.AI's GLM as the mandatory reasoning component. On 24 April 2026, the organizers issued an announcement relaxing that requirement: *"Due to persistent issues with the Z.AI API, we would like to inform you that usage of the Z.AI API is no*

longer mandatory. However, all teams are still required to integrate an API in your project. You may use any available alternatives such as Gemini, GPT, or other relevant APIs."


[Vice Director] Muhammad Hariz 12:00 PM
Dear @Participant

Due to persistent issues with the Z.AI API, we would like to inform you that **usage of the Z.AI API is no longer mandatory**

However, all teams are **still required to integrate an API in your project.** You may use any available alternatives such as Gemini, GPT, or other relevant APIs.

Please make the necessary adjustments accordingly. For those who already submitted, please feel free to make changes on your GitHub before the submission deadline.

If you have any questions, feel free to reach out. Thank you for your understanding.
📌 @Committee @Administrator @PIC

ClaimIQ's glm_client module is designed as a provider-agnostic abstraction that selects the LLM endpoint at runtime via environment variables. The current production configuration uses **ILMU's nemo-super model** (a Malaysian-hosted, OpenAI-API-compatible LLM provider) with full backward compatibility for Z.AI's GLM via the same client. The module name glm_client is preserved for codebase continuity; throughout this document, references to the **"LLM Reasoning Service Layer"** refer to this provider-agnostic abstraction.

1.2 Background

Malaysia has approximately **5,000+ panel GP clinics under PMCare alone**, with comparable populations across other major TPAs (MediExpress, Healthmetrics, ePesakit). Most are small private practices — SMEs employing 1–3 admin staff — who depend on TPA reimbursements as their primary revenue stream. On the TPA side, claims teams must process millions of submissions annually with limited tooling for explainability or fraud anomaly detection.

The economic friction in this ecosystem is **not high outright rejection rates** (PMCare publicly reports approximately 1.3% denial), but the operational overhead and cashflow drag on both sides:

- **For TPA operations:** manual triage of borderline claims, fraud detection across high-volume submissions, and providing decision rationale to corporate clients are labor-intensive.
- **For SME clinics:** the query-and-resubmission cycle extends payment cycles from ~30 days to ~60 days, adds 10+ minutes of admin work per query, and risks claims becoming unpayable if the 14-day resubmission deadline is missed. For a clinic submitting 200 claims/month, this compounds into significant cashflow drag and weeks of delayed RM payments.

ClaimIQ addresses both sides by providing decision intelligence over the full claim lifecycle — from clinic-side submission to TPA-side adjudication — with an LLM doing context-aware reasoning that no traditional rules engine can replicate, while deterministic safety gates enforce auditability and prevent autonomous incorrect outcomes.

Previous Version

There is no prior public version of ClaimIQ. The closest existing systems are TPA portals (e.g., Mediline) which act as **submission and tracking interfaces**. They do not provide AI-powered risk scoring, conversational claim explanation, fraud anomaly detection at the line-item level, or portfolio-level intelligence generation.

Changes in Major Architectural Components

ClaimIQ introduces three architectural categories not present in incumbent tooling:

1. A **provider-agnostic LLM Reasoning Service Layer** (glm_client) that centralizes prompt construction, retry logic, structured-output parsing, and graceful fallback.
2. An **8-step deterministic pipeline** (claims_processor) that combines rules-based validation with LLM-driven reasoning at well-defined gates, with a **"Safety Freeze"** that prevents autonomous adjudication.
3. A **Multi-Modal Evidence Pipeline** that uses MedGemma (Google's medical Vision Language Model running on a separate inference endpoint) to extract structured data from X-rays, lab reports, and invoices before the LLM reasons over the structured representation.

1.3 Target Stakeholders

Stakeholder	Role	Expectations
TPA Operations Analyst (Primary User – Console)	Reviews adjudication recommendations, monitors fraud alerts, generates portfolio reports, handles denials and referrals	Clear decision rationale, fast triage, anomaly visibility, AI-generated weekly intelligence reports, traceable audit timeline per claim
Clinic Admin / GP (Primary User – Portal)	Views submitted claims, receives advisory on denials, files appeals, monitors clinic-level performance	Plain-language status updates, BM/EN bilingual UI, AI-drafted appeal rebuttals, transparent reasoning
Clinic Owner / Doctor	Reviews portfolio-level performance, approves process changes	Visibility into cashflow impact, denial trends, time saved

Development Team	Builds and maintains the platform	Modular code, clear API contracts, isolated LLM integration for prompt versioning, schema migrations system
QA Team	Validates LLM output quality, system behavior, hallucination handling	Defined acceptance criteria for AI outputs, prompt regression suite, deterministic safety gates

2. System Architecture & Design

2.1 High-Level Architecture

Overview

Type	Details
System	Single-binary FastAPI application serving both API endpoints and a vanilla JavaScript single-page UI as static assets
Architecture	Modular monolith with seven cooperating Python modules; LLM Reasoning Service Layer cleanly isolated for provider-portability
Deployment	Currently single-instance deployment (FastAPI + uvicorn). Frontend assets bundled with backend; no separate frontend service. SQLite for relational persistence; FAISS for policy vector index

ClaimIQ runs as a **single FastAPI application** that serves both REST API endpoints (under /api/...) and a vanilla JavaScript single-page application (under /) as static assets from execution/frontend/. The deliberately minimal frontend stack — vanilla HTML/CSS/JS, no build step, no framework — was chosen to maximize iteration speed during the hackathon window and to make the entire system deployable from a single Python process.

The backend is internally organized into **eight cooperating Python modules** with distinct responsibilities. They ship as one process (modular monolith) but have clean module boundaries and could be split into independent services post-MVP without rewriting business logic.

Module	File	Responsibility
API Server	api_server.py	FastAPI route handlers, request validation (Pydantic), CORS, error mapping

Claims Processor	claims_processor.py	Orchestrates the 8-step pipeline; enforces Safety Freeze and fraud-override logic
LLM Reasoning Service Layer	glm_client.py	All LLM interaction. Provider abstraction (ILMU/Z.AI), prompt construction, JSON-mode structured output, retry with backoff, auth-failure cooldown, intelligent mock fallback for non-prod environments
Database Layer	database.py	SQLite schema, 17-state claim lifecycle, schema migrations, audit logging, processing locks, analytics queries
Policy RAG Engine	rag_engine.py	FAISS vector index (sentence-transformers, MiniLM-L6-v2 embeddings) for semantic policy lookup; assembles policy context for LLM adjudication prompts
Claim Scrubber	claim_scrubber.py	Deterministic pre-adjudication validation (ICD-10 format, duplicate detection, required fields)
Eligibility Engine	eligibility_engine.py	Member coverage check (deductible, copay, coinsurance, annual outpatient limits, waiting periods, prior-auth requirements)
Cross-Reference Engine	cross_reference_engine.py	Deterministic checks comparing parsed evidence (lab values, invoice line items) against doctor's clinical description;

		surfaces critical contradictions before LLM reasoning
Evidence Parser	evidence_parser.py + document_triage.py	Calls MedGemma vision model to extract structured data from X-rays, lab reports, invoices; image quality gating
EOB Generator	eob_generator.py	Produces Explanation of Benefits document after adjudication

LLM as Service Layer (Critical Design Decision)

The LLM is not treated as a generic "AI module" bolted onto the application. It is encapsulated in glm_client.py with the following responsibilities:

1. **Provider Abstraction.** _get_provider_config() resolves credentials at runtime from environment variables (ILMU_API_KEY / ZAI_API_KEY, ILMU_BASE_URL / ZAI_BASE_URL, ILMU_MODEL / ZAI_MODEL). The OpenAI-compatible Python client is constructed against whichever provider is configured. This allows the codebase to switch between ILMU (nemo-super), Z.AI's GLM, or any future OpenAI-API-compatible provider without code changes.
2. **Structured-Output Prompting.** All decision-making prompts are issued in JSON mode (response_format={"type": "json_object"}) with explicit schema instructions in the system prompt. Robust output parsing strips markdown code fences if the model emits them.
3. **Retry with Exponential Backoff.** Transient failures retry up to 3 times with exponentially-increasing sleep (1s, 2s, 4s). Authentication failures (401, "token expired") are detected separately and trigger a 15-minute cooldown to prevent thrashing against an expired key.

4. **Production-vs-Development Behavior.** When APP_ENV=prod, terminal LLM failures raise GLMServiceUnavailable and the pipeline fails closed (claim → REFERRED for human review). In dev/demo environments, terminal failures fall back to deterministic mock responses keyed off the system prompt — preserving demo flow while making production behavior strict.
5. **Intelligent Mock Fallback.** The _get_intelligent_mock() function returns realistic JSON responses keyed off prompt patterns, enabling local development and CI without burning API tokens. Mocks are clearly logged (logger.error("All GLM retries failed. Using intelligent fallback mock for hackathon demo.")) so they never silently substitute for a real call in production.

2.1.2.1 Prompt Construction & Context Window Design

Each LLM call in ClaimIQ is constructed dynamically using a structured context window consisting of three parts:

- **System Prompt** – Defines the role of the model (e.g., adjudication engine, fraud detector) and enforces output format (JSON-only)
- **User Prompt** – Task-specific instruction
- **Context Window** – Injected structured data from upstream pipeline components

Context Window Composition by Pipeline Step

1. Claim Data Extraction

- Raw clinical notes (unstructured text input)

2. Clinical Validation

- Extracted structured claim data
- Parsed evidence (from MedGemma / evidence_parser)
- Cross-reference results

3. Medical Coding

- Structured claim data (diagnosis, procedures, medications)

4. Adjudication (RAG + GLM)

- Structured claim data
- Parsed evidence (lab reports, X-rays, invoices)
- Cross-reference engine results
- Retrieved policy clauses (Top-K from FAISS via rag_engine.get_policy_context())
- Claim financial details (amounts, billing breakdown)

This is implemented in:
execution/glm_client.py → adjudicate_claim() glm_client.py

5. Fraud Detection

- Full claim object (structured)
- Cross-reference inconsistencies
- Derived features (amount anomalies, contradictions)

6. GP Advisory

- Final decision output
- Claim data
- Fraud results

Response Handling

All LLM responses:

1. Are returned in **JSON format (response_format=json_object)**

2. Are cleaned and parsed in _call_glm()
3. Are passed to:
    - claims_processor.py for pipeline continuation
    - Database for persistence
    - Frontend for display

2.1.2.2 Token Management, Chunking & Truncation Strategy

To ensure reliability within LLM token limits, ClaimIQ implements controlled context construction before each GLM call.

1. Policy Context (RAG)

- FAISS retrieves **Top-K = 5** policy documents
- If combined context is too large:
    - Only highest-relevance documents are retained
- Implemented in:
  execution/rag_engine.py rag_engine.py

2. Evidence Data Handling

- Evidence parsed via MedGemma is filtered to:
    - Key findings (lab values, diagnoses)
    - Important clinical indicators
- Non-essential metadata is excluded before LLM input

3. Structured JSON Optimization

- Only relevant fields are included in prompts
- Internal system fields (e.g., _fallback_fields) are removed before sending to GLM

4. Token Limits

- max_tokens is enforced at **4096** in _call_glm()
- Large inputs are implicitly controlled by:
    - Selective field inclusion

- RAG Top-K filtering

5. Fallback Behavior

If token or API failures occur:

- System retries (up to 3 attempts)
- Falls back to mock responses (non-production)
- Forces manual review in production

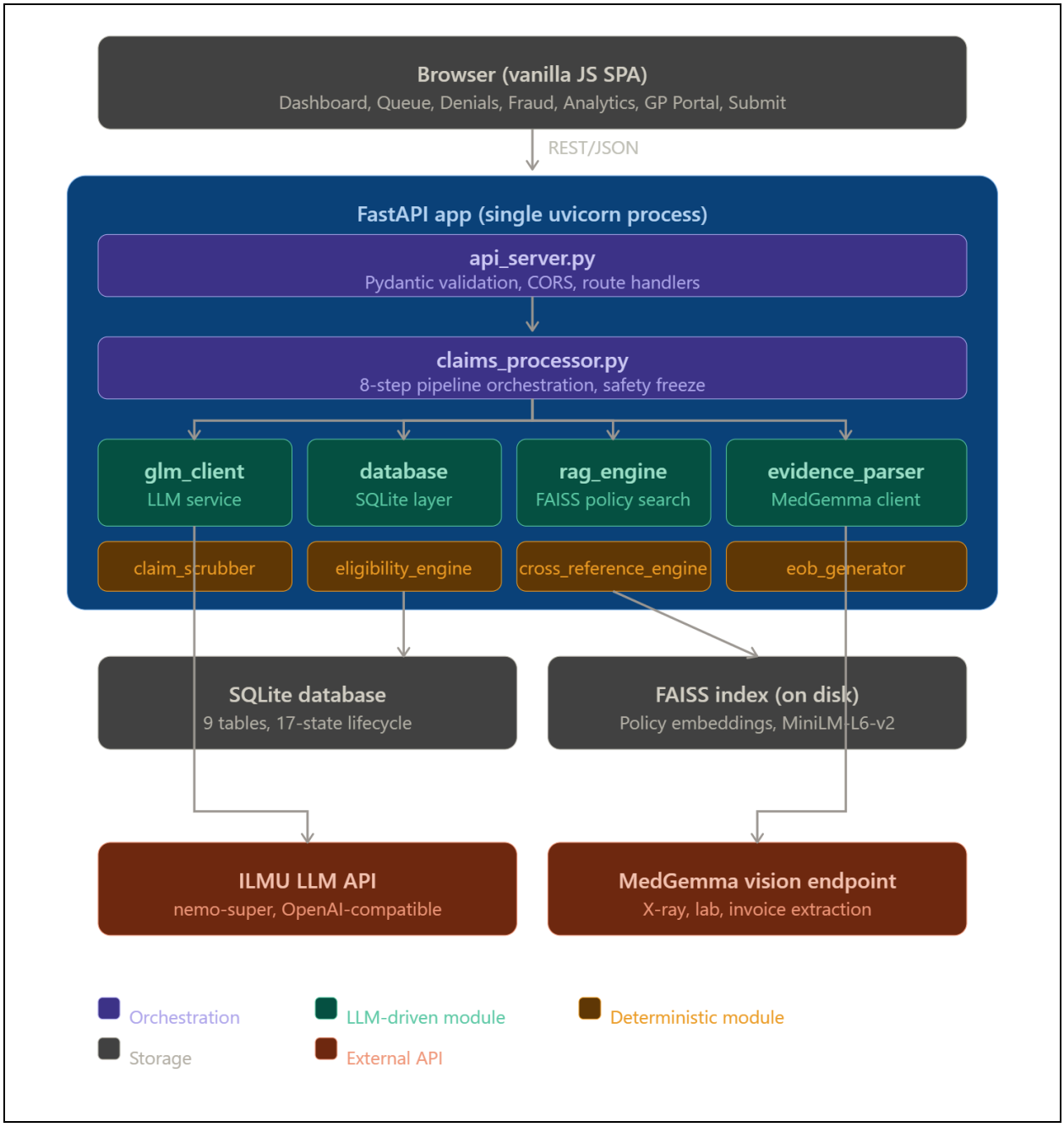
This service-layer approach is what makes the LLM **load-bearing**: every meaningful decision the user receives — adjudication recommendation, fraud risk explanation, conversational Q&A, weekly report narrative, GP advisory — passes through this layer. Removing it disables the core reasoning surfaces of the product.

Safety Freeze (Critical Design Decision)

A unique architectural decision in ClaimIQ is the explicit **Safety Freeze**: the pipeline never autonomously confirms an APPROVED or DENIED final state. Both outcomes are converted to **REFERRED** (queued for human review) before being persisted as the final decision. The LLM's recommendation is recorded — including its reasoning, confidence, and any fraud overrides — but human sign-off is the only path to a binding adjudication.

This converts the system from "autonomous AI adjudicator" (high regulatory risk, hard to defend) into "decision-support assistant" (clearly within Domain 2's *"recommendation of actions or strategies"* scope and *"explanation of decisions in a clear and user-understandable manner"* requirement). It also protects the team from claims of practicing TPA authority — the LLM advises, the human decides.

Dependency Diagram



Key external API calls:

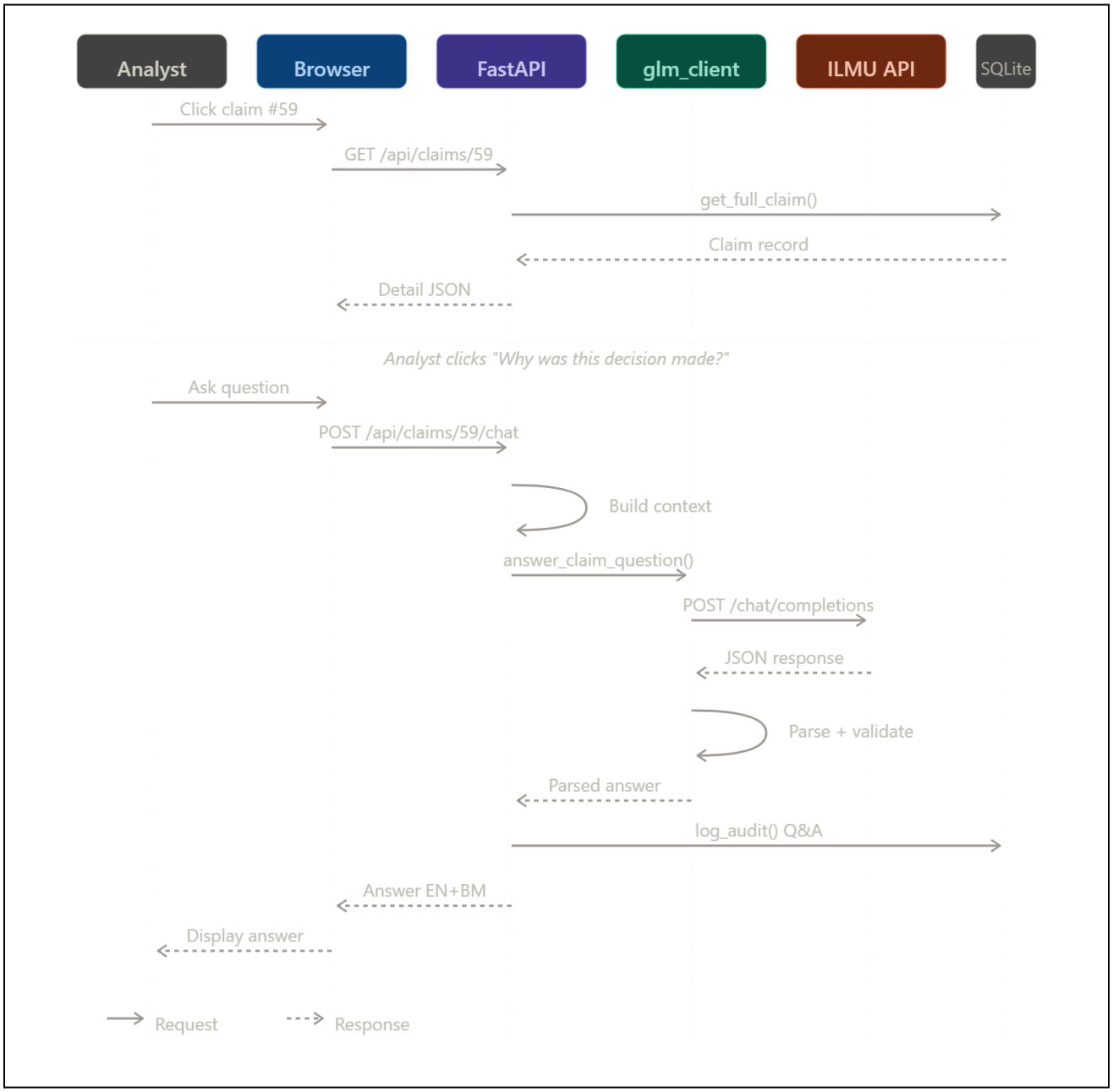
Caller	Callee	Endpoint / Action

Browser SPA	FastAPI	POST /api/claims/submit — intake new claim with optional evidence base64
Browser SPA	FastAPI	POST /api/claims/process/{claim_id} — kick off the 8-step pipeline
Browser SPA	FastAPI	GET /api/claims/{claim_id} — full claim detail (extraction, decision, fraud, advisory, EOB)
Browser SPA	FastAPI	POST /api/claims/{claim_id}/chat — conversational Q&A about a claim ("Ask GLM about this claim")
Browser SPA	FastAPI	POST /api/claims/{claim_id}/appeal — file an appeal; LLM drafts rebuttal in EN+BM
Browser SPA	FastAPI	GET /api/analytics/kpis — clean claim rate, denial rate, AR days, auto-screened %
Browser SPA	FastAPI	GET /api/analytics/weekly-report — LLM-generated weekly intelligence narrative
Browser SPA	FastAPI	GET /api/analytics/fraud-heatmap — fraud risk distribution + flagged claims
Browser SPA	FastAPI	POST /api/demo/seed — populate synthetic demo dataset

glm_client	ILMU API	POST /v1/chat/completions (model=nemo-super, response_format=json_object)
evidence_parser	MedGemma endpoint	POST /analyze — image base64 → structured JSON (findings, impressions, extracted lab values)
rag_engine	(in-process)	FAISS semantic search over pre-built policy index (sentence-transformers MiniLM-L6-v2)
claims_processor	SQLite	Direct sqlite3 access via database.py helper functions

2.2 Sequence Diagram — "Ask GLM About This Claim" Flow

This is the most distinctive demonstration of LLM-powered decision intelligence in ClaimIQ: a TPA analyst opens a denied or referred claim and asks “Why was this decision made?” — the system grounds the answer in the specific claim’s evidence, decision history, and clinic context.



This sequence represents the primary system functionality: end-to-end claim adjudication.

This sequence demonstrates the key mechanics: contextual data gathering from the persisted claim record, prompt construction with the LLM Service Layer, structured JSON output with

EN+BM bilingual responses, retry/backoff resilience, and audit logging of every Q&A interaction.

Actors

- Clinic Admin (User)
- API Server
- Claims Processor
- GLM Service Layer
- RAG Engine
- Database

Flow

1. Clinic Admin → submits claim (POST /claims/submit)
2. API Server → stores claim in database
3. User → triggers processing (/process/{id})
4. API → calls process_claim() in claims_processor.py

Pipeline Execution

5. Claims Processor:
    - Extract claim data (GLM)
    - Parse evidence (MedGemma)
    - Cross-reference validation
    - Scrubbing (rule engine)
    - Eligibility check
    - Coding (GLM)
    - Adjudication (GLM + RAG)
    - Fraud detection (GLM)
    - GP advisory generation (GLM)
6. Decision is evaluated:

    - Fraud gate applied
    - Safety Freeze applied
    - Final status → REFERRED
7. Results stored in database:
    - Decision
    - Fraud score
    - Advisory
    - Audit logs
8. Analyst retrieves results via API

2.3 Technological Stack

Layer	Technology	Justification
Frontend	Vanilla HTML/CSS/JavaScript (no framework, no build step)	Maximum iteration speed during the hackathon window; avoids npm/build pipeline complexity; ships from the same Python process as the backend
Backend	FastAPI (Python 3.10+)	Async support for parallel LLM calls; automatic OpenAPI documentation; Pydantic schema validation aligns directly with structured-output LLM parsing
Database	SQLite (with schema migrations system in database.py)	Zero-config for hackathon demo; supports the 17-state claim lifecycle;

		clear migration path to Postgres for multi-tenant production
LLM	**ILMU nemo-super** (primary, OpenAI-compatible API at api.ilmu.ai/v1); Z.AI GLM (compatible fallback via same client)	Selected after the organizers' 24 April 2026 announcement relaxed the Z.AI requirement; ILMU provides reliable Malaysian-hosted inference with strong JSON-mode structured output
Embeddings	sentence-transformers all-MiniLM-L6-v2	Lightweight (80MB), CPU-friendly, sufficient for the policy corpus size; used by the RAG engine
Vector DB	FAISS (faiss-cpu)	In-process, no separate service; Inner-product index over normalized embeddings (cosine similarity)
Vision Model	Google MedGemma 4B	Medical-domain VLM for X-ray/lab/invoice extraction; runs on a HuggingFace/Colab inference endpoint via ngrok tunnel; called from medgemma_client.py
HTTP Client	openai Python SDK (used as a generic OpenAI-compatible client)	Same SDK works against ILMU and Z.AI; minimal code surface

Auth (MVP)	None (single-tenant demo)	Out of scope for hackathon MVP; production would use Clerk/Auth0 with role claims
Hosting (target)	Single Python process on any VPS or Railway/Render	Single binary makes deployment trivially portable

2.4 Key Data Flows

ClaimIQ handles three primary data flow patterns:

1. **Claim adjudication flow** — moves a single claim through the 8-step pipeline, combining deterministic gates and LLM reasoning. Persisted to SQLite at every step.
2. **Conversational Q&A flow** — short-lived interactions in the "Ask GLM about this claim" chat. Audit-logged but not part of adjudication state.
3. **Analytics & reporting flow** — aggregations over the claims table for KPIs (clean claim rate, denial rate, AR days), fraud heatmaps, and the LLM-generated AI Weekly Intelligence Report.

Data Flow Diagram (Level-1, described)

External Entities:

[TPA Analyst] [Clinic Admin / GP] [ILMU LLM API] [MedGemma Vision API]

Processes:

(1) Validate & Persist Claim

(2) Parse Evidence (MedGemma)

(3) Cross-Reference Evidence vs Description (deterministic)

(4) Scrub & Check Eligibility (deterministic)

(5) LLM Pipeline: Extract → Validate → Code → Adjudicate (with RAG) → Fraud → Advisory

(6) Apply Safety Freeze + Fraud Override (deterministic)

(7) Generate EOB

(8) Answer Conversational Claim Questions ("Ask GLM")

(9) Generate Weekly Intelligence Report

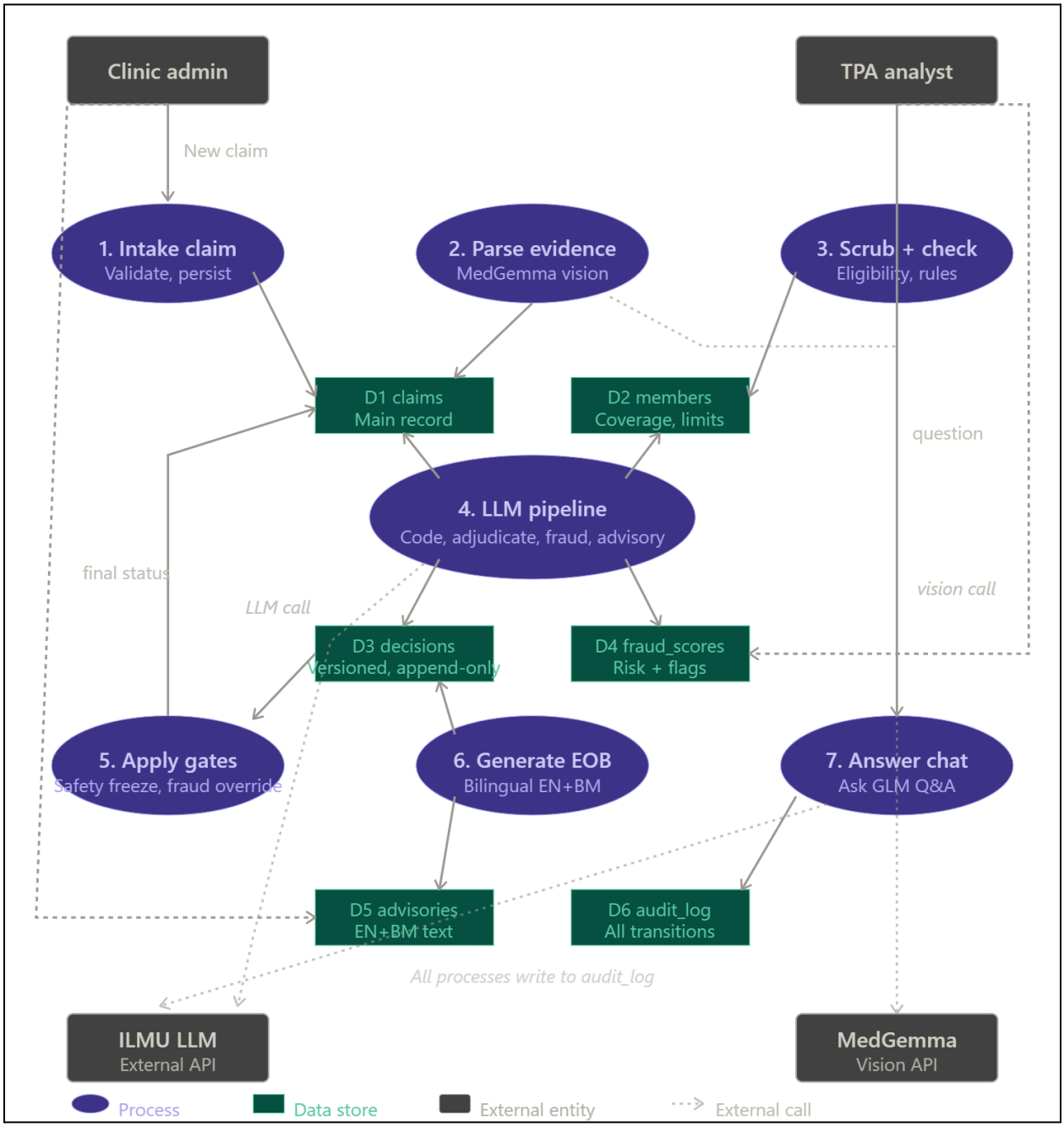
(10) Process Appeal (LLM-drafted rebuttal)

Data Stores:

D1: claims	— main claim record (extracted, coded, status, etc.)
D2: members	— insurance member records (coverage, deductible, limits)
D3: decisions	— versioned adjudication outputs (per processing run)
D4: fraud_scores	— fraud risk scores and flags
D5: advisories	— GP-facing advisory text (EN + BM)
D6: eobs	— Explanation of Benefits records
D7: appeals	— appeal submissions and LLM-drafted rebuttals
D8: audit_log	— actor + action + status transitions per claim

D9: carc_codes — denial reason code reference table

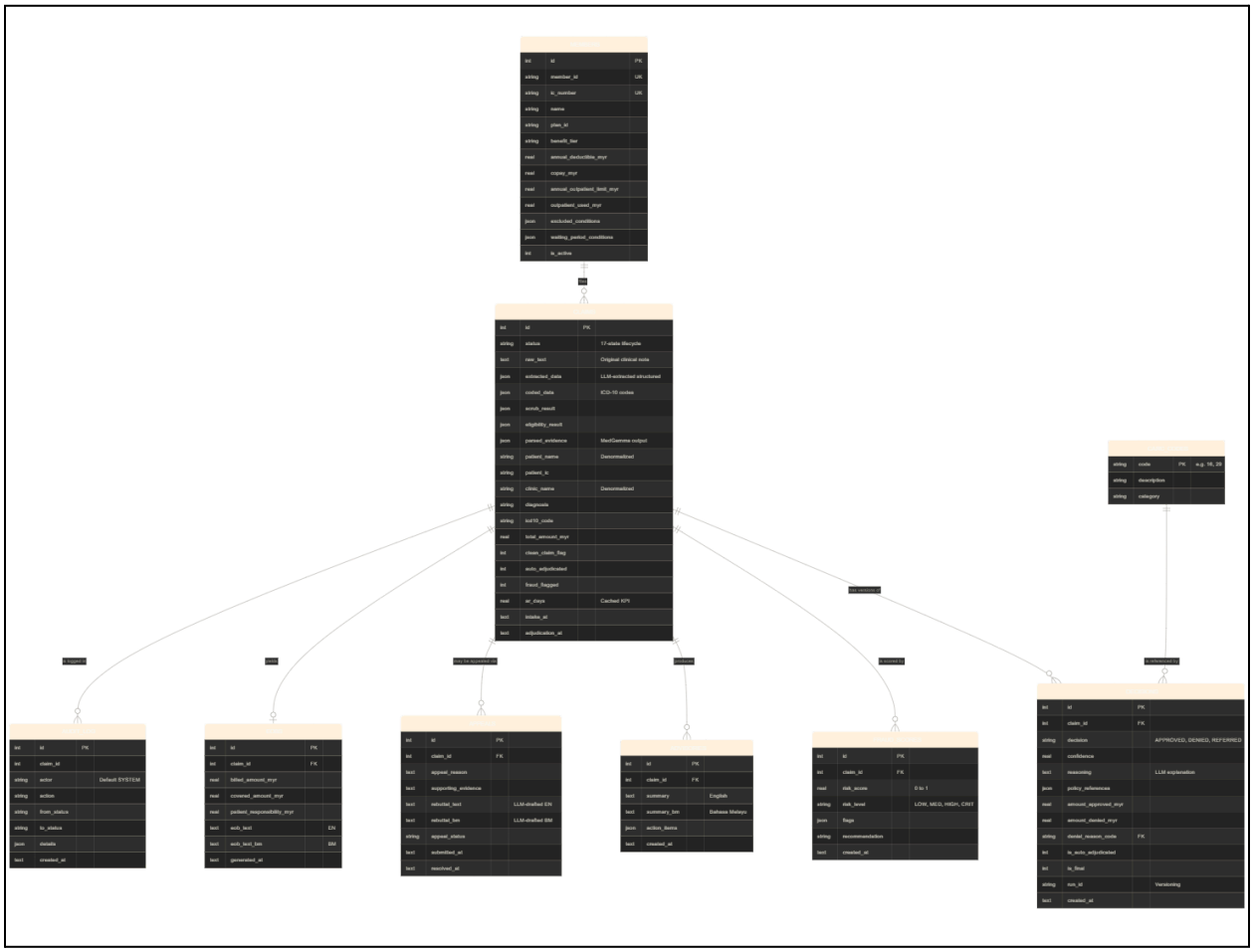
D10: policy_index — FAISS index of policy excerpts (on disk)



DFD compliance: No black holes (every store has both reads and writes), no miracles (every output traces to inputs), processes are verb-phrases, stores are noun-phrases.

Database Schema (3NF, 9 core tables)

The schema implements a 17-state claim lifecycle (vs. the typical 5-state model in similar systems), with denormalized patient/clinic fields on the claims table for fast queries and normalized child tables for audit-able artifacts.



Textual-based

```
claims                                  -- Main claim record (denormalized fast-path fields)
  ├── id (PK)
  ├── status              -- one of 17 lifecycle states
  ├── lifecycle_stage             -- mirrors status; separate to allow lifecycle-level filtering
  ├── raw_text                    -- the original submitted clinical note
  ├── extracted_data (JSON)   -- structured output from LLM extraction
```

```
├── coded_data (JSON)      -- ICD-10 / CPT codes assigned by LLM
├── scrub_result (JSON)
├── eligibility_result (JSON)
├── parsed_evidence (JSON)  -- MedGemma vision output
├── cross_ref_result (JSON)
├── patient_name, patient_ic, patient_age, patient_gender   -- denormalized
├── clinic_name, clinic_id                                  -- denormalized
├── diagnosis, icd10_code
├── visit_date, filing_date, total_amount_myr
├── consultation_fee_myr, medication_fee_myr, procedure_fee_myr
├── clean_claim_flag, auto_adjudicated, fraud_flagged, requires_prior_auth
├── intake_at, scrub_at, eligibility_at, adjudication_at, payment_at, closed_at
├── ar_days, cycle_time_hours       -- derived KPIs cached for analytics
└── created_at, updated_at

members                             -- Insurance member records
├── id (PK)
├── member_id (UNIQUE), ic_number (UNIQUE)
├── name, date_of_birth, gender
├── plan_id, benefit_tier
├── coverage_start_date, coverage_end_date, is_active
├── annual_deductible_myr, deductible_met_myr
├── copay_myr, coinsurance_pct, max_out_of_pocket_myr
├── annual_outpatient_limit_myr, outpatient_used_myr, max_per_visit_myr
├── excluded_conditions (JSON), waiting_period_conditions (JSON)
├── requires_prior_auth_conditions (JSON)
└── created_at

decisions                           -- Versioned adjudication outputs (append-only)
├── id (PK), claim_id (FK)
├── decision, confidence, reasoning
```

```
├── policy_references (JSON)
├── amount_approved_myr, amount_denied_myr, patient_responsibility_myr
├── denial_reason_code, denial_reason_description
├── conditions, appeal_recommendation, full_result (JSON)
├── is_auto_adjudicated, is_final
└── run_id, created_at

fraud_scores                    -- Append-only fraud assessments
├── id (PK), claim_id (FK)
├── risk_score (0-1), risk_level
├── flags (JSON), recommendation, full_result (JSON)
└── created_at

advisories                      -- GP-facing advisory (bilingual)
├── id (PK), claim_id (FK)
├── summary, summary_bm
├── action_items (JSON), full_result (JSON)
└── created_at

appeals                         -- Clinic-side appeals + LLM rebuttals
├── id (PK), claim_id (FK)
├── appeal_reason, supporting_evidence
├── rebuttal_text, rebuttal_bm
├── appeal_status, appeal_decision, appeal_reasoning
└── submitted_at, resolved_at

eobs                            -- Explanation of Benefits
├── id (PK), claim_id (FK)
├── billed_amount_myr, covered_amount_myr, patient_responsibility_myr
├── denial_code, denial_description
├── eob_text, eob_text_bm
```

```
└── generated_at

audit_log                          -- Every state transition + significant action
├── id (PK), claim_id
├── actor (default 'SYSTEM')
├── action, from_status, to_status
├── details (JSON)
└── created_at

carc_codes                         -- Denial reason code reference table
├── code (PK)              -- e.g., '16', '29', 'B7'
├── description            -- e.g., 'Claim/service lacks information'
└── category           -- e.g., 'Documentation', 'Timely Filing', 'Provider'
```

The schema is organized around claims as the central entity, with five append-only artifact tables (decisions, fraud_scores, advisories, appeals, eobs) capturing what the system *did* to each claim, and three reference/support tables (members, audit_log, carc_codes) providing context, traceability, and standardization.

Central entity: claims. Every record in the system traces back to a row here. The table deliberately denormalizes a few fast-path fields — patient_name, clinic_name, diagnosis, total_amount_myr — so the dashboard's claim list and analytics queries can render without joining JSON blobs. The full structured representation lives in JSON columns (extracted_data, coded_data, parsed_evidence, scrub_result, eligibility_result) that retain everything the LLM and deterministic gates produced. The 17-state status field tracks the claim through its full lifecycle from INTAKE to PAID or CLOSED, with intermediate states like SCRUB_FAILED, REFERRED, and APPEAL_APPROVED reflecting real TPA workflows.

Append-only artifact tables. Five tables — decisions, fraud_scores, advisories, appeals, eobs — are kept separate from claims rather than merged as columns. This is deliberate: each of these is produced by a versioned LLM prompt, and historical outputs must be preserved for

explainability, prompt regression analysis, and audit defense. decisions in particular uses a run_id and is_final flag so a claim can have multiple decision records across processing runs, with only one marked final at any given time. This makes "show me the full decision history of this claim" trivially queryable, which matters for both judge-facing demos and downstream regulatory review.

members table. Captures real insurance benefit semantics — annual deductibles, copays, coinsurance, annual outpatient limits, waiting-period and prior-auth conditions stored as JSON arrays. The eligibility_engine reads this table during Step 2 of the pipeline to deterministically check whether a member's coverage permits the claim, before any LLM reasoning runs. Keeping eligibility deterministic (rules over a structured schema) rather than LLM-driven is intentional: coverage limits are exact arithmetic, and the LLM should not be making calls about "does this person have RM 80 of outpatient budget left."

audit_log table. Every meaningful state transition writes a row here, regardless of whether it was caused by a user action, an LLM decision, or a deterministic gate. This gives the system a single source of truth for "what happened to claim #59 and when," which surfaces in the Audit Timeline panel of the claim detail UI. Decoupling audit from the artifact tables means we never have to reconstruct history from scattered timestamps.

carc_codes reference table. Standardizes denial reason codes (CARC = Claim Adjustment Reason Code, a US-origin convention adapted to Malaysian TPA terminology). Codes like 16 (claim lacks information), 29 (filing deadline exceeded), and B7 (provider not eligible) are referenced by decisions.denial_reason_code so denial reasons are consistent across the entire portfolio, not free-text strings the LLM invents per claim. This consistency is what makes the dashboard's "denial reasons breakdown" chart possible.

Design decisions worth highlighting:

- **No ORM.** Direct sqlite3 access via helper functions in database.py. Faster for the team to ship in 18 hours and removes a layer of magic — every SQL query is visible and reviewable.

- **Schema migrations system.** Three numbered migrations (claim enrichment, decision versioning, eligibility consumption tracking) are applied idempotently on startup and recorded in _migration_history. The schema is designed to evolve, not be reset.
- **JSON for nested structured data, columns for query-able fields.** Hybrid approach — patient name needs to be query-able (it's denormalized as a column), but the full extracted claim data is too irregular to flatten and is kept as JSON.
- **Bilingual fields are paired columns.** summary and summary_bm, rebuttal_text and rebuttal_bm, eob_text and eob_text_bm — both languages generated in a single LLM call and persisted side-by-side, so the EN/BM toggle in the UI is a free field swap, not a second LLM round-trip.

3. Functional Requirements & Scope

3.1 Minimum Viable Product (MVP)

The 8 core features built and demoed for the Preliminary Round:

Feature	Description
Claims Intelligence Dashboard	Real-time TPA operations view with KPIs (Clean Claim Rate, Denial Rate, Avg AR Days, Auto-Screened %), denial-reasons breakdown, claims-by-status donut, recent claims feed. Powered by direct SQL aggregations + LLM-generated AI Weekly Intelligence Report.
8-Step Adjudication Pipeline	The core value flow: Extraction (LLM) → Evidence Parsing (MedGemma) → Cross-Reference (deterministic) → Scrub + Eligibility (deterministic) → Clinical Validation (LLM) → Coding (LLM) → Adjudication with RAG (LLM) → Fraud Detection (LLM) → GP Advisory (LLM) → EOB Generation.
Claim Detail with "Ask GLM" Chat	Per-claim drill-down showing extraction, decision, fraud, advisory, EOB, and audit timeline. The "Ask GLM about this claim" chat lets analysts query the specific claim's record with suggested prompts ("Why was this decision made?", "What is the fraud risk based on?", "How can I avoid denials for this?"). All responses are bilingual EN+BM.
Fraud Detection Page	Cross-referenced fraud signals: deterministic evidence contradictions (e.g., "Doctor claims severe dengue but lab shows normal platelets") combined with LLM-driven anomaly

	explanation. Risk distribution view (Low / Medium / High / Critical) and a list of high-risk flagged claims.
GP Clinic Portal (Bilingual EN/BM)	Clinic-side view with claims summary tiles (Total Submitted / Approved / Denied / Pending), recent claims & status feed, and an Appeal action that triggers LLM-drafted rebuttal generation in both languages.
Claims Queue & Submit	Claim intake supporting raw clinical-note text + optional X-ray/invoice base64 upload. Filterable queue view across all claims with status tagging.
Safety Freeze	Architectural feature: every claim that the LLM marks APPROVED or DENIED is automatically converted to REFERRED with explanatory reasoning appended. Ensures human sign-off and protects against autonomous incorrect outcomes.
Seed Demo Utility	One-click Seed Demo button populates the system with synthetic Malaysian-format claims (clinic names, ICD-10 codes, RM amounts, member records) so judges and demo viewers see a populated, realistic-looking system without manual data entry.

3.2 Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
LLM Latency	LLM-mediated decisions return within reasonable demo bounds (observed p95 ~5–7s per call against ILMU).	_call_glm() enforces max_tokens=4096 per call; pipeline runs steps sequentially but each step is independently timed and logged for diagnostic visibility.
Cost Efficiency	Token usage capped per call; identical prompts can be cached via the prompt-fingerprint pattern (post-MVP).	temperature=0.3 for stable structured output; response_format=json_object reduces re-prompts; max_tokens=4096. Mock fallback in dev avoids burning tokens during testing.
Reliability	LLM calls succeed for ≥95% of valid claim records; transient failures retry; auth failures degrade gracefully without thrashing.	Retry with exponential backoff (1s, 2s, 4s) up to 3 attempts. Auth failures (401/expired) trigger 15-min cooldown via _AUTH_FAILURE_UNTIL global. Production env raises GLMServiceUnavailable and the pipeline forces the claim to REFERRED. Dev env falls back to _get_intelligent_mock().

Explainability	Every LLM-driven output includes human-readable reasoning, not just a numeric score.	All decision prompts request a reasoning field. Frontend never renders confidence/risk without explanation. The Audit Timeline on the claim detail page shows every state transition.
Bilingual Output	GP-facing advisory text and appeal rebuttals are available in both English and Bahasa Melayu.	A single LLM call returns both translations in one structured JSON response, persisted side-by-side (summary + summary_bm, rebuttal_text + rebuttal_bm). EN/BM toggle in UI flips field, no second LLM call.
Data Privacy & Synthetic Data	No real PHI/PII; all demo data is synthetic.	"Seed Demo" generates synthetic Malaysian-format records. Demo IC numbers follow real-world format but are randomly generated. Production deployment would require formal data-sharing agreements with TPAs.
Maintainability & Migrations	Schema can evolve without manual DB resets.	database.py runs idempotent numbered migrations on startup, tracked in _migration_history. New migrations added as functions, never as ad-hoc SQL.
Auditability	Every meaningful action is logged and reproducible.	audit_log table records every status transition with actor, action, from/to status, and details. Decisions are append-only with run_id and is_final fields, so a claim's full decision history is preserved across reruns.

Safety	The system cannot autonomously approve or deny claims.	Safety Freeze logic in claims_processor.py converts every APPROVED → REFERRED and every DENIED → REFERRED with explanatory reasoning, regardless of LLM confidence. Documented and tested.

3.3 Out of Scope / Future Enhancements

The "Grand Vision" items deliberately deferred from the Preliminary Round MVP:

- **Direct TPA system integrations.** Live API connectors to PMCare's Mediline portal, MediExpress, Healthmetrics, and ePesakit. MVP operates over synthetic data and a stub submission flow.
- **Production-grade authentication.** SSO, MFA, and per-clinic authorization scopes. MVP runs as single-tenant demo without auth.
- **Multi-instance deployment with shared cache.** MVP runs as a single uvicorn process; production deployment requires Redis-backed prompt cache and Postgres backend.
- **Mobile-native applications.** Currently responsive vanilla-JS web only.
- **Full PDPA compliance review.** Required pre-production but out of MVP scope.
- **Predictive cashflow forecasting per clinic.** Backlog item — would extend the Analytics module with time-series forecasting.
- **Real-time TPA query subscription.** Webhook-driven query notifications from TPA backends. MVP uses on-demand refresh.
- **Drug-formulary lookup against the Malaysian National Pharmaceutical Regulatory Agency (NPRA) database.** MVP uses a curated synthetic drug list.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation

Grayscale Rollout & A/B Testing

For post-hackathon production deployment, ClaimIQ would follow a phased release strategy. New LLM prompt versions or new pipeline steps are released to 10% of traffic first with shadow logging — the new prompt's output is recorded but the user still sees the existing version's output. After 48 hours of monitoring (validation accuracy, latency, hallucination flags from QA spot-checks), the rollout expands to 50%, then 100%. A/B testing then compares decision-acceptance rates and analyst time-to-decision between prompt versions.

Emergency Rollback (ER) & Golden Release

A "golden release" is the most recent version that passed all CI checks and was stable in production for at least 7 days. Emergency rollback is triggered automatically by the priority matrix below or manually by the engineering lead. Rollback redeploys the golden release; prompt-template rollbacks are even faster since they require only a config redeploy.

Priority Matrix

Priority	Trigger Condition	Action
P0 — Critical	LLM API failure rate > 10% over 5 min, OR any user-visible 5xx error rate > 1%	Immediate emergency rollback to golden release. Page engineering lead. Production env automatically forces affected claims to REFERRED state via GLMServiceUnavailable handling.
P1 — High	LLM hallucination flag rate (schema-validation	Auto-revert affected prompt to previous version. Notify QA. The Safety Freeze provides a

	failures + manual QA flags) > 5% over 1 hour	backstop — no autonomous bad outcomes can ship.
P2 — Medium	p95 LLM latency > 10s for 10 min	Throttle non-critical LLM calls (weekly report generation), preserve interactive flows (chat, detail view). Investigate.
P3 — Low	Auth failure cooldown engaged (signals expired key)	Notify on-call to rotate credentials. System remains operational against mock fallback in dev; production raises GLMServiceUnavailable.

Monitoring

The MVP includes lightweight in-process monitoring sufficient for hackathon evaluation, with a documented path to production-grade observability:

- **Logs:** Structured Python logging with module-level loggers (claimiq.glm, claimiq.processor, claimiq.eligibility, etc.). Every LLM call logs the model name, attempt number, response time, and token count. Production target: ship to Grafana Loki or Datadog.
- **Metrics:** Per-pipeline-step timing recorded on each claim's result["steps"]. KPIs (clean claim rate, denial rate, AR days) are stored as derived columns and recomputed on each claim update for fast analytics queries.
- **Alerts:** Threshold alerts mapped to the Priority Matrix above. Production target: PagerDuty.
- **Health checks:** GET /api/health returns service liveness and database connectivity status.

4.2 Assumptions

Key operational and environmental conditions assumed valid during development and deployment:

- TPA analysts and clinic admins access ClaimIQ from desktop or tablet browsers with stable internet connectivity.
- The LLM API endpoint (currently ILMU nemo-super) remains available with valid credentials throughout the demo period.
- The MedGemma vision endpoint (HuggingFace/Colab via ngrok tunnel) is reachable when claims include image evidence; if unreachable, evidence parsing degrades to "no evidence available" without breaking the pipeline.
- All demo data is **synthetic**. No real PHI/PII is stored, processed, or transmitted at any point. Patient names, IC numbers, and clinic names in seed data follow Malaysian format conventions but are randomly generated.
- TPA query categories and denial reason codes (CARC) used in prompts and the database are derived from publicly available standards (US TPA conventions adapted for Malaysian context, plus PMCare's published GP working guidelines).
- Clinic users are literate in either English or Bahasa Melayu. Other Malaysian languages (Mandarin, Tamil) are out of scope for the MVP.
- The Preliminary Round demo runs on the team's deployed Vercel/Railway environment or locally via python execution/api_server.py.

4.3 External Dependencies

Tool	Purpose	Risk
ILMU LLM API (nemo-super)	Core reasoning engine — handles extraction, clinical validation, coding, adjudication, fraud detection, advisory, conversational Q&A, weekly reports, appeal rebuttals	**High.** API rate-limiting, outage, or schema drift would disable all decision-intelligence features. **Mitigation:** retry with exponential backoff (3 attempts), auth-failure cooldown, intelligent mock fallback in dev, Safety Freeze ensures no autonomous bad outcomes, production env raises GLMServiceUnavailable and forces REFERRED state.
MedGemma Vision Endpoint	Multi-modal evidence extraction from X-rays, lab reports, invoices	**Medium.** Endpoint is hosted on Colab via ngrok and may go down. **Mitigation:** evidence parsing failures are caught and logged; the pipeline proceeds with whatever structured data is available, and Step 3.6 (Evidence Quality Gate) flags missing evidence as a CRITICAL pipeline finding.
FAISS + sentence-transformers	Local in-process semantic search over policy corpus	**Low.** Pure local computation; no network dependency. Index is rebuilt by build_policy_index.py on demand.

GitHub	Source of truth for code	**Low.** Outages historically rare and short.
Vercel / Railway / hosting target	Deployment	**Low/Medium.** Single-process app means easy redeploy; can run on any VPS or PaaS that supports Python.
Python ecosystem (FastAPI, openai SDK, pydantic, sqlalchemy)	Standard web/data stack	**Low.** Pinned versions in requirements.txt; no exotic dependencies.

5. Project Management & Team Contributions

5.1 Project Timeline

The Preliminary Round development window for ClaimIQ was effectively a single intense sprint. Key phases:

Day	Date	Activities
Day One	Wednesday 22nd April 2026	Problem validation against public sources (CodeBlue Nov 2025 reporting on PMCare's ~1.3% denial rate, PMCare's published GP working guidelines on the 14-day filing deadline and query workflow). Domain framing: clinic-as-SME, cash-flow optimization. Architectural decisions: dual-portal frontend (TPA Console + GP Portal), modular monolith backend, the Safety Freeze design as the answer to the "decision support, not autonomous AI" requirement. Initial database schema sketch (17-state lifecycle, 9-table design with append-only audit artifacts).
Day Two	Thursday 23rd April 2026	Core backend build: FastAPI app structure, SQLite schema

		and migrations, deterministic gates (claim_scrubber, eligibility_engine, cross_reference_engine). LLM Reasoning Service Layer (glm_client.py) with provider-agnostic configuration, JSON-mode structured output, retry logic, and the intelligent mock fallback for dev. Initial prompt design across the 5 LLM-driven pipeline steps (extraction, validation, coding, adjudication, fraud detection).
Day Three	Friday 24th April 2026	8-step pipeline orchestration in claims_processor.py with the Safety Freeze and fraud-override logic. MedGemma vision endpoint integration (evidence_parser.py, document_triage.py) for X-ray, lab, and invoice extraction. FAISS-based policy RAG (rag_engine.py) with sentence-transformer embeddings. Vanilla-JS frontend: Dashboard, Claims

		Queue, Denials, Fraud Detection, Analytics, Submit, and the bilingual GP Portal. "Ask GLM about this claim" chat UI on the claim detail view. Seed Demo utility with synthetic Malaysian dataset. **Provider pivot:** following the organizers' announcement relaxing the Z.AI mandatory requirement, the team migrated to ILMU nemo-super — possible without code changes thanks to the provider-agnostic LLM client.
Day Four	Saturday 25th April 2026	End-to-end integration testing, AI Weekly Intelligence Report generation, EOB generation, appeal workflow with bilingual LLM-drafted rebuttals. Screen recording of the complete user journey. Documentation streams running in parallel: PRD (Sohaib), SAD (Ivy), TAD/QA (Lee + Sadiq). Pitch deck design (Anas).

Day Five	Sunday 26th April 2026	Final document review, PDF export, README finalization with video link, GitHub commit, submission by 7:59 AM.

5.2 Team Members & Roles

Member	Role	Responsibilities
Muhammad Anas Adil	Lead Engineer & Pitch	End-to-end system development: Fast API backend and 8-step pipeline orchestration, LLM Reasoning Service Layer (glm_client.py), all prompt design, deterministic gates (Scrubber, Eligibility, Cross-Reference, Safety Freeze), SQLite schema and migrations, vanilla-JS frontend (Dashboard, Claims Queue, Fraud, GP Portal, "Ask GLM" chat UI), MedGamma vision integration, FAISS RAG index, demo seed data. Also led pitch deck design.
Mohammad Sohaib Saim	Product Requirements (PRD)	Product Requirements Document - problem definition, target users. User stories, MVP scope, success metrics, and product roadmap.
Ivy Neema Wairi	System Analysis (SAD)	This System Analysis Documentation - architecture,

		data flows, schema, NFR, monitoring strategy, and alignment between documentation and the built product.
Lee Xian Xin & Ahmed Mohamed Sadiq	Testing Analysis (TAD) & QA	Testing Analysis Documentation and quality assurance strategy - risk matrix (likelihood * severity), test planning across the 8-step pipeline (happy-path, negative-path, NFR validation)c, AI-specific testing (prompt regression, oversized input handing, hallucination guards), and overall test execution.

5.3 Architectural Recommendations

For continuation beyond the Preliminary Round, the following architectural improvements are recommended in priority order:

1. **Migrate from SQLite to PostgreSQL** before any multi-tenant deployment. SQLite's single-writer constraint will not scale beyond a single demo instance. The schema migration system already in place makes the swap mechanical.
2. **Introduce Redis for prompt-fingerprint caching.** Identical LLM prompts within a 24-hour window should return cached results to manage cost. The MVP defers this; production will need it under any non-trivial load.
3. **Promote the LLM Reasoning Service Layer to an independent microservice** once a second consumer (e.g., a mobile clinic app, or a TPA back-office reporting service) needs LLM reasoning. The clean module boundary in glm_client.py makes this a mechanical extraction.
4. **Add a prompt regression test suite.** Every prompt version should be evaluated against a fixed corpus of claims with expected JSON output ranges — this catches reasoning regressions when prompt templates evolve.
5. **Implement prediction-vs-outcome reconciliation** once real TPA adjudication data is available (post-pilot), enabling continuous accuracy measurement and prompt fine-tuning.
6. **Move MedGemma off Colab/ngrok.** Acceptable for hackathon demo; production needs a stable inference endpoint (HuggingFace Inference Endpoints, Modal, or self-hosted GPU instance).
7. **Frontend framework adoption.** Vanilla JS was the right call for hackathon speed; production complexity will warrant migration to a framework (Next.js or SvelteKit) for component reuse, state management, and type safety.